Contents

API Blitz — Site Specification **1**
1. Overview . . . 1
2. Tech stack . . . 1
3. Configuration (set once, no code changes) . . . 2
4. Data model . . . 2
5. Authentication . . . 3
6. Team registration flow . . . 3
7. Site map . . . 4
8. Submissions . . . 4
9. Real-time features (Supabase Realtime) . . . 5
10. Feature set . . . 5
11. Build order (5 to 6 days) . . . 5
12. Inputs needed from you before launch . . . 6

API Blitz — Site Specification

Revision 2. Single source of truth for the registration, team-showcase, and submission platform for API Blitz.

What changed in this revision

- **No second-member confirmation.** When the leader submits the form, both people are on the team immediately (pending admin approval only).
- Added a **WhatsApp community button**.
- Resolved all open items: editable submissions until deadline, global timer, domain-only eligibility, showcase shows team and member names only, no phone numbers.
- Data model simplified: member rows are keyed by email, so the one-team rule holds even before the second person has ever logged in.

---

1. Overview

Goal: Manage team registration, team showcase, API allotment, and submissions for API Blitz, a 2-person-team API event.

Out of scope: Judging/scoring (handled separately, later).

---

2. Tech stack

Layer	Choice	Notes
Frontend	Next.js (App Router) + Tailwind CSS	One codebase for public pages, dashboard, admin
Auth	Auth.js (NextAuth), Google provider	`hd: "dau.ac.in"` restricts sign-in; also re-validated server-side (the client param is spoofable)
Database	PostgreSQL via Supabase	Also provides Realtime and Storage
ORM	Prisma	Type-safe schema and migrations
File storage	Supabase Storage	Rulebook PDF

Layer	Choice	Notes
Hosting	Vercel (app) + Supabase (DB/storage/realtime)	Free tiers are enough at this scale
Email (optional)	Resend	Confirmation emails on approval/submission

---

3. Configuration (set once, no code changes)

Setting	Purpose
`ADMIN_EMAILS`	`microsoftclub@dau.ac.in` + a test email
`EVENT_START_AT`	Global timestamp: countdown target and API reveal moment
`SUBMISSION_DEADLINE`	Global timestamp: submissions lock after this
`WHATSAPP_COMMUNITY_URL`	Target of the WhatsApp button
Rulebook PDF	Static file in storage, linked from landing page and dashboard

---

4. Data model

```
User
  id
  email            (unique, @dau.ac.in only, stored lowercase)
  name             (from Google)
  role             enum [participant, admin]

Team
  id
  name             (unique, case-insensitive)
  status           enum [pending_approval, approved, rejected]
  createdAt

TeamMember
  id
  teamId           -> Team
  email            (UNIQUE, lowercase - the one-team-per-person rule)
  name             (typed by leader; overwritten with Google name on first login)
  userId           -> User, nullable (filled when that person first logs in)
  isLeader         boolean

Api
  id, name, docsLink

ApiAssignment
  teamId -> Team, apiId -> Api   (5 rows per team, inserted by admin)

Submission
  id
  teamId           (UNIQUE - one submission per team)
```

`githubUrl`
`submittedAt`
`updatedAt`

Why `TeamMember.email` is the unique key (not `userId`): the second member may not have logged in yet, so there is no `User` row to constrain. Keying on email means nobody can be put on two teams, regardless of whether they have ever signed in. It is enforced by the database, so the UI cannot be bypassed.

Why `Submission.teamId` is unique: a second submit is an update to the team's one row, not a new row. Duplicates are impossible by construction.

---

5. Authentication

- Google OAuth only, restricted with `hd=dau.ac.in`.
- **Server-side re-check is mandatory:** after the callback, confirm the email ends in `@dau.ac.in` before creating or loading a `User`.
- Email format is `<roll_number>@dau.ac.in` (for example `202501069@dau.ac.in`).
- Admins are determined by `ADMIN_EMAILS` at login. No roles UI is needed.
- No other eligibility rules: any `@dau.ac.in` account may participate.

---

6. Team registration flow

1. **Leader logs in with Google.** Name and email are auto-filled and locked.
2. **If the leader's email is already in `TeamMember`** (for example, someone else already added them), skip the form and show that team's page instead.
3. **Leader fills the form:**
   - Team name
   - Member name
   - Member email
4. **Validation on submit** (all inside one database transaction):
   - Member email ends in `@dau.ac.in`
   - Member email is not the leader's own email
   - Neither email is already in `TeamMember`
   - Team name is not already taken (case-insensitive)
   - Unique-constraint violations from simultaneous submissions are caught and shown as normal errors
5. **Team is created immediately** with status `pending_approval`. Two `TeamMember` rows are written: the leader (linked to their user) and the member (`userId` empty for now).
6. **The second member needs to do nothing.** When they log in later, the system matches their Google email to their `TeamMember` row, links their account, replaces the typed name with their real Google name, and shows the same team page. A dismissible banner reads "You've been added to [Team] by [Leader]" for transparency; it requires no action.
7. **Edits:** until the admin approves, the leader can edit the team name, member name, and member email. Every edit re-runs the validations in step 4. After approval these are locked and changes go through an admin.
8. **Admin approves or rejects.**
   - Approved: both members see the team dashboard.
   - Rejected: the team's rows are released so both people and the team name are free to register again (assumed default; see section 12).

Known trade-off: because there is no member confirmation, a leader can add a classmate without their agreement. Mitigation: the admin queue shows both names and emails for review, and an admin can reject or fix a team.

---

7. Site map

Public (no login)

- **Landing page:** event description, rulebook link, countdown to `EVENT_START_AT`, login button, anonymized live activity ticker
- **Team showcase gallery:** approved teams only, showing **team name and member names only**

Logged-in participant, no team

- Registration form (section 6)

Logged-in participant, team pending approval

- Status screen with team details, the leader's edit controls, and a "Under review by organizers" message

Logged-in participant, team approved

- **Dashboard** (identical for both members, keyed by `teamId`):
  - Team name and both members
  - 5 assigned APIs (hidden behind "revealed when the event starts" until `EVENT_START_AT` passes)
  - Submission form (GitHub link)
  - Rulebook link
  - Countdown to `SUBMISSION_DEADLINE`
  - Progress tracker: Registered, Approved, Submitted

On every logged-in screen

- **"Join WhatsApp Community" button** linking to `WHATSAPP_COMMUNITY_URL`. It is shown only to logged-in `@dau.ac.in` users, so the invite link is not exposed publicly. (Easy to also add to the landing page if you prefer.)

Admin (`microsoftclub@dau.ac.in` + test email)

- **Approval queue:** approve or reject pending teams, with both members' names and emails visible
- **API assignment:** searchable list of the 100 APIs, assign 5 per team, with a per-API "assigned count" to balance variety
- **Submissions table:** all teams, GitHub links, timestamps, CSV export
- **Announcements:** editable text block for rule clarifications

---

8. Submissions

- One submission per team. Submitting again **edits** the existing link.
- **Editable until `SUBMISSION_DEADLINE`**, then locked.
- The lock is enforced **on the server** by comparing the current time to the deadline, not only by disabling the button in the UI.
- Validation: must be a well-formed `https://github.com/<owner>/<repo>` URL.
- Both teammates see and can edit the same submission.
- The API list is hidden until `EVENT_START_AT`. The submission form stays open throughout the event.

---

9. Real-time features (Supabase Realtime)

Where	What updates live	Anonymity
Landing page	"A team just submitted" / "New team registered" ticker	Fully anonymous
Admin approval queue	New registrations appear without refresh	N/A (admin only)
Team dashboard	Flips automatically on approval or when event start passes	N/A (own team only)

Anonymity is enforced at the data level. The public channel broadcasts a generic payload such as `{ event: "submission_made", timestamp }` and never the raw `Submission` or `Team` row, so team names cannot be seen in the browser's network tab.

Skipped intentionally: real-time on the API-assignment screen.

---

10. Feature set

Base build

- Everything in sections 5 to 9
- Progress tracker, anonymized ticker, public showcase, CSV export, WhatsApp button

Stretch goals

- Email confirmations (Resend)
- Discord/WhatsApp webhook to the organizer channel on key events
- Admin analytics (registrations over time, approval funnel, submissions vs deadline)
- API reveal "unlock" animation

Out of scope

- Judging/scoring module
- Phone numbers or any extra contact fields

---

11. Build order (5 to 6 days)

Day	Work
1	Auth (Google OAuth, `hd` restriction, server-side re-check), Prisma schema, config, deploy skeleton to Vercel
2	Registration form, validations and transaction, email-matching on first login, edit-until-approval, admin approval queue
3	Admin API-assignment UI, landing page with countdown and rulebook, showcase gallery, WhatsApp button
4	Team dashboard (API reveal, submission form with deadline lock), anonymized realtime ticker

Day	Work
5	Admin submissions table and CSV export, edge cases (rejected teams, email edits, leader added as someone else's member), responsive polish
6	Buffer: bug fixes and testing with real test accounts on real devices

---

12. Inputs needed from you before launch

1. **Exact timestamps:** event start and submission deadline (with timezone, presumably IST)
2. **WhatsApp community invite link**
3. **Rulebook PDF**
4. **The 100 APIs:** name and docs link for each (a spreadsheet is fine), and the 20+ API sets you plan to assign
5. **Your test email** to add to `ADMIN_EMAILS`
6. **Confirm rejected-team behavior:** the assumed default is that a rejected team is fully released so both people can register again. Say so if you want rejections to be final instead.